Chapter 1: Introduction to Visual C++ 6.0

In this course you will learn to develop MFC-based applications using Visual C++® version 6.0.

This course assumes that you have a fundamental knowledge of Visual C++ and MFC. It builds on this knowledge to enable you to build advanced applications, such as OLE DB consumers, ADO-based database applications, and Internet-based applications.

(CD-ROM displays the following tip.)

If you are not familiar with Visual C++ and MFC, you should first complete the course *Mastering MFC Fundamentals Using Visual C++*.

This chapter discusses the architecture of Visual C++ version 6.0 and describes the enhancements that have been made to Visual C++ and MFC in this version.

Objectives

After completing this chapter, you will be able to:

- List some major features of Visual C++ and MFC Library.
- List the major enhancements to Visual C++ and MFC in Visual Studio™ 6.0.
- Create, build, and run an MFC-based application.

13 Overview of Visual C++ and MFC

The MFC Library and Visual C++ provide an environment that you can use to create a wide variety of applications. This section provides an overview of the features of Visual C++ and MFC. It also describes the steps that you follow to create a simple MFC-based application with MFC AppWizard.

This section includes the following topics:

- Features of Visual C++
- Features of the MFC Library
- Creating an MFC-based Application

Notes

14 Features of Visual C++

Visual C++ helps you develop applications that address diverse business needs. It offers a wide variety of features and tools to help you develop applications using the latest technologies, such as MFC and the Component Object Model (COM).

Features of Visual C++

Features of Visual C++ include:

- Native COM support

  Visual COM editing, location- and language-independent COM browsing, MFC templates, and compiler support simplify component-based development.
- Visual Schema Designer

  Enhance your database application development productivity by remotely analyzing the schema of a Microsoft® SQL Server™ database from your desktop.
- Microsoft Transaction Server

  Microsoft Transaction Server (MTS) helps you build high-performance transaction-based applications.
- Active document server support

  Visual C++ provides wizard and programmatic support for creating active document servers that can be invoked in any active document container, such as Microsoft Office 97 and Internet Explorer 4.0.

Visual C++ Development Environment

Visual C++ provides an integrated environment for developing applications. Using Visual C++, you can create, test, refine, and maintain your applications.

The development environment of Visual C++ includes:

- A project workspace to help manage your projects.
- Tools for building projects.
- Editors for designing user interface and creating resources.
- Shared development components, including an optimizing compiler, an incremental linker, and an integrated debugger.

For a complete list of new features and enhancements in Visual C++ 6.0, search for "What's New for Visual C++ Version 6.0" in MSDN™ Library Visual Studio 6.0.

Editions of Visual C++

Visual C++ is available in three editions: Standard, Professional, and Enterprise.

Standard Edition

The Standard Edition contains all the features of the Professional Edition, except code optimizations, the Profiler, and static linking to the MFC Library.

Notes

Professional Edition

The Professional Edition enables you to develop and distribute commercial-quality software products. Features include MFC, the Active Template Library (ATL), the Component Object Model (COM), and OLE DB.

Enterprise Edition

The Enterprise Edition provides tools and components for building and validating enterprise-level COM applications. This edition contains all the features of the Professional Edition, and also includes Microsoft Transaction Server, SQL Editor and Debugger, Visual SourceSafe™, and support for ActiveX® Data Objects (ADO) and Remote Data Objects (RDO).

Notes

15 Features of the MFC Library

The MFC Library is a collection of C++ classes and an application framework designed primarily for creating Microsoft Windows®-based applications. This collection of classes extends the C++ language to include most of the basic structural elements that are used to create Windows-based applications. The application framework defines the structure of an application and handles many routine tasks for the application.

The MFC Library is built on top of the Win32® application programming interface (API). This API is a set of functions that the operating system exposes for use by applications. MFC exposes base classes that represent common objects in the Windows operating system, such as windows and menus.

MFC does not encapsulate, or wrap, the entire API. It wraps just the main structural components and commonly used components. Because MFC is written in C++, you can easily use the Win32 API to make native calls to the operating system.

Benefits of Using MFC

The MFC Library enables you to build on the work of expert Windows developers. MFC shortens development time, makes code more portable, provides support without reducing programming freedom and flexibility, and provides easy access to hard-to-program user-interface elements and technologies such as ActiveX and Internet programming. MFC simplifies database programming through OLE DB Templates and ADO, and simplifies network programming through Windows Sockets. MFC makes it easy to program such features as property sheets, print preview, and floating toolbars.

MFC offers you many advantages when developing full-featured Windows applications, database applications, ActiveX controls and components, and Internet applications, such as:

- An application framework on which you can build a Windows application.
- Compatibility with previous MFC versions and the new C++ classes.
- The largest base of reusable C++ source code in the industry.
- Integration with Visual C++.

Document/View Architecture

The MFC document/view architecture provides a single, consistent way of coordinating application data, referred to as a document, and views of that data. A document is a data object with which the user interacts in an editing session. A view is a window object through which the user interacts with the document.

MFC Class Hierarchy

MFC provides several classes that serve a wide range of programming needs. The first step in programming with MFC is to become familiar with these classes and learn how they are related to one another in the MFC hierarchy. Some classes are used directly; others serve as base classes for the classes that you create.

Notes

When learning about the MFC classes, it is helpful to organize them into categories. Some important categories of MFC classes are:

- Application architecture classes
- User-interface classes
- Database classes
- Internet classes

Message Maps

Before you start writing MFC applications, it is important to understand how MFC maps messages to the functions that will handle the messages. The application framework implements the message map data structure, which provides the link between the message ID and the function that will handle the message. Each entry consists of a message-specific macro. Standard Windows messages have predefined macros containing an implicit ID and handler name. Command message macros contain an explicit ID and handler name.

Below is an example of a message map for the **CMyView** class with two entries. Note that the base class is included to allow the framework to continue searching for a given handler if one does not exist within **CMyView**. Every standard Windows message has a macro of the form ON_WM_*xxx*, where *xxx* is the name of the message. A simple convention is used to generate the handler function name. The name of the function starts with "On", followed by the name of the message with "WM_" removed and only the first letter of each word capitalized. In the following example, the handler name for the first message would be **OnCreate**:

```
BEGIN_MESSAGE_MAP(CMyView, CView)
        ON_WM_CREATE()
        ON_COMMAND(ID_APPLY_SEQUENCE, OnApplySequence)
END_MESSAGE_MAP()
```

You can add a message map entry by using ClassWizard or WizardBar in Developer Studio®. You can also add the entry manually.

Notes

16 Creating an MFC-based Application

Visual C++ helps you build applications that use the MFC Library. Visual C++ integrates development tools, such as wizards and editors, to provide flexibility to meet various programming needs.

To see a demonstration of how to create a simple MFC-based application by using Visual Studio, click this icon.

(CD-ROM plays the demonstration, "Creating an MFC-based Application.")

MFC AppWizard

MFC AppWizard automates the steps in creating the basic framework of a project. You choose the options you want for your project, and MFC AppWizard creates the project files and adds skeleton code. You can refine or enhance the code added by MFC AppWizard to implement custom requirements.

Workspace Window Panes

The Workspace window provides three views of a project, each in a separate pane:

- ClassView

  Lists the classes that you created to implement the application.
- FileView

  Lists the header and implementation files that constitute the project.
- ResourceView

  Lists the resources that are used in the project.

ClassWizard

ClassWizard simplifies using classes in the MFC Library by performing various routine programming tasks. Using ClassWizard, you can create and implement classes, add member functions and variables to a class, and map messages to functions.

Components and Controls Gallery

The Component and Controls Gallery contains reusable code, such as Visual C++ components, ActiveX controls, and components created by third-party vendors. When you insert a component into a project, the class definitions and code associated with the component are added to the project. You can then add code to implement the functionality of these components.

Notes

17 Enhancements to Visual C++

This section describes some major enhancements to the development environment in Visual C++ 6.0.

This section includes the following topics:

- Debugger
- Editor
- Projects
- Wizards
- Tools

Notes

18 Debugger

Visual C++ 6.0 includes several new features that simplify the debugging process.

For a complete list of new debugger features, search for "What's New for Visual C++ Version 6.0, Debugger" in MSDN Library Visual Studio 6.0.

Edit and Continue

The Edit and Continue feature enables you to edit code during a debugging session without having to quit the session and rebuild the application. Changes made during a session are simply recompiled and applied to the executing application in memory.

For more information about Edit and Continue, see Using the Edit and Continue Feature in Chapter 2: Debugging and Error Handling.

GetLastError() Value Displayed as Register

A new pseudo-register, ERR, displays the last error code for the current thread. ERR is displayed as a hardware register, though actually it is not. You can view the contents of this pseudo-register in the Watch window or **Quickwatch** dialog box by using the standard procedures for displaying registers.

To display the error code in meaningful form, use the hr format specifier with ERR as shown:

```
ERR, hr
```

AfxDumpStack for Diagnosing Applications

The **AfxDumpStack** function generates an image of the current stack and dumps that image to the debug output device, such as **afxdump**. You can use this function in debug and non-debug versions of the MFC libraries to get diagnostic information, even if the debugger is not installed on the end user's computer.

New Formatting Symbols

The following table lists some new formatting symbols.

Symbol	Description
hr	Displays 32-bit values in a more meaningful way, such as **S_OK** or **E_NOTIMPL**.
st	Displays string values as ANSI or Unicode.
wm	Displays numeric values decoded into the names of windows message numbers (WM_ constants).

Variants Decoded for Display

The debugger automatically displays variants in their correct form. For example, integers are displayed numerically, and BSTRs are displayed as strings. The debugger also displays the type of the variant.

Notes

19 Editors

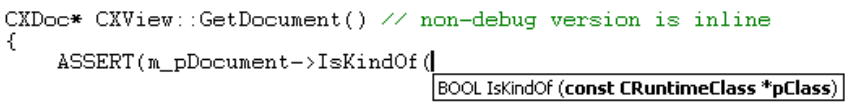
The enhancements to the Visual C++ Editor make it easier to write code for your applications.

For a complete list of new editor features, search for "What's New for Visual C++ Version 6.0, Editors" in MSDN Library Visual Studio 6.0.

Automatic Statement Completion

The text editor in Visual C++ 6.0 provides the Automatic Statement Completion feature, which uses IntelliSense® technology to help you write code for your application. When you type characters, such as "**.**" or "**->**", IntelliSense presents information that helps you complete the code statement or function call.

The following illustration shows the IntelliSense information that appears when you type " **(** " after a function name.



Parameter Pop-Ups

In the source editor, you can get type information about a parameter by placing the mouse cursor on the parameter. Similarly, you can see the function definition by placing the cursor on a function.

Resource Editor Support for Internet Explorer 4.0 Controls

The resource editor provides support for Internet Explorer 4.0. To access the Internet Explorer 4.0 wrapper classes, you use the **Controls** toolbar in the dialog editor to add these controls to your forms and dialog boxes.

Notes

20 Projects

Some key enhancements relating to building projects in Visual C++ are described below.

For a complete list of new project features, search for "What's New for Visual C++ Version 6.0, Projects" in MSDN Library Visual Studio 6.0.

Dynamic Parsing

When you create a class, variable, or member function, Visual C++ updates ClassView and WizardBar to display the new items, without requiring you to save changes first.

Invoking Dialog Editor from ClassView and WizardBar

You can now go directly to the dialog editor from a class that implements a dialog box by choosing **GoTo Dialog Editor** from ClassView or WizardBar.

New Form Command

The **New Form** command generates a form and is available from the **Insert** menu, ClassView, and WizardBar. This command provides ClassWizard functionality for **CFormView**, **CDialog**, and the database view classes. In addition to this functionality, the **New Form** command also creates a dialog box with all the appropriate options.

Notes

21 Wizards

Some key enhancements to wizards, such as MFC AppWizard, are described below.

For a complete list of new wizard features, search for "What's New for Visual C++ Version 6.0, Wizards" in MSDN Library Visual Studio 6.0.

Enhancements to MFC AppWizard

MFC AppWizard offers several new options for projects that you create in Visual C++ 6.0, including:

- Single-document interface or multiple-document interface applications that are not based on the document/view architecture. This enables streamlined use of MFC for applications that are not document-based.
- Toolbars that work like Internet Explorer rebar controls.
- Active document containment.
- Windows Explorer-style applications.

Enhancements to ClassWizard

ClassWizard now enables you to add messages and variables to Internet Explorer 4.0 controls. You can also derive new classes based on MFC Internet Explorer 4.0 control classes.

For more information about these control classes, see Internet Explorer 4.0 Common Controls in Chapter 5: Using Controls.

Enhancements to the ATL Object Wizard

The ATL Object Wizard contains a new object category named Data Access, which you can use to create OLE DB consumers and providers.

For more information about building consumer applications, see Chapter 7: Using OLE DB Templates for Data Access.

Notes

22 Tools

This topic provides a brief overview of some tools that are new in Visual C++ 6.0, such as Component Manager and Visual Analyzer. You will also look at enhancements to the Test Container tool.

For a complete list of new tools features, search for "What's New for Visual C++ Version 6.0, Tools" in MSDN Library Visual Studio 6.0.

Component Manager

Note Component Manager is supported only in the Enterprise Edition of Visual C++.

The Component Manager is a tool that enables you to store objects and share them between software tools and custom reusable components. The Component Manager improves support in the following areas:

- Reuse

  Support for cataloging and locating relevant designs, code and services.
- Dependency tracking

  Support for establishing and querying relationships between objects.
- Tool interoperability

  Support for publishing standardized descriptions of systems, which allows independent tools to exchange data.
- Data resource management

  Global metadata for an enterprise data warehouse, and a resource library of available services and components.
- Team development

  Support for managing concurrent activity on different versions and configurations of application design and development.

Visual Studio Analyzer

Note Visual Studio Analyzer is supported only in the Enterprise Edition of Visual C++.

Visual Studio Analyzer server-side components provide information about the Microsoft components, such as COM, ADO, and MTS, running on a particular computer. You can use this information with the Visual Studio Analyzer client to retrieve information about a distributed application, such as what it is doing, how it is performing, and where problems are occurring.

Notes

Enhancements to Test Container

Improvements to Test Container include:

- Support for OCX 96 features, such as windowless controls, inactive controls, flicker-free activation, and quick activation.
- Visual Basic® Scripting Edition (VBScript) support for writing test scripts.
- Selective enabling/disabling of container features to simulate different container environments.
- Selective logging of events, property changes, and property edit requests.

Notes

23 Enhancements to MFC

This section describes some enhancements to MFC that help you build custom applications that access data sources and the Web.

This section includes the following topics:

- Database
- Internet

Notes

24 Database

The two major enhancements in MFC for data access are OLE DB Templates and ActiveX Data Objects (ADO). These enhancements enable you to build powerful database applications.

OLE DB Templates

OLE DB is the latest data access methodology from Microsoft. This methodology enables you to access any kind of data source by using a set of standard data access interfaces.

Using OLE DB directly from Visual C++ involves writing a large amount of code. OLE DB template classes make OLE DB much easier to use while providing excellent performance.

For more information about OLE DB Templates, see Chapter 7: Using OLE DB Templates for Data Access.

The new MFC class, **COleDBRecordView**, provides the functionality of OLE DB templates within the context of the MFC document/view architecture.

ActiveX Data Objects

ADO simplifies database-related operations, such as connecting to a database and retrieving rowsets. MFC supports ADO in the following ways:

- ADO Data Bound Dialog Wizard

  This wizard guides you through the process of creating a data-bound dialog box with ADO.

- ADO data control

  This control enables you to access data exposed through an OLE DB data provider application.

- Data-bound controls

  The data-bound controls, such as **DataGrid** and **DataList**, help you process the data retrieved by the data control. These controls have a **DataSource** property and can be bound to a data control. You can include these controls in your dialog boxes to display the retrieved data.

For more information about ADO, see Chapter 8: Creating ADO Database Applications.

Notes

25 Internet

This topic provides a brief overview of some Internet-related enhancements in MFC. These enhancements help you build applications that access the Internet.

Dynamic HTML Control with CHtmlView

CHtmlView is a wrapper class for the Web Browser ActiveX control. The Web Browser control enables your application to display HTML pages just as a Web browser would display them. **CHtmlView** provides many browser features, including a history list, favorites (bookmarks), and security features.

For more information about using **CHtmlView**, see Implementing HTML Views in Chapter 9: Building Internet Applications.

Updates to Existing MFC Classes

The following table describes improvements to existing MFC classes that provide Internet functionality.

Class	Description
CHttpServer	Includes a function for writing data to the server.
CHttpServerContext	Includes functions that provide control over the size of chunks written to the HTTP server.
CInternetSession	Supports setting cookies for a specified URL, returning cookies for a specified URL and all its parent URLs, and retrieving the variable specifying the length of the cookie stored in the buffer.

Notes